

UNIVERSIDADE FEDERAL DE SÃO PAULO – UNIFESP

Instituto de Ciência e Tecnologia
Disciplina de Visão Computacional

CAUÊ EBERSPACHER MENEZES

DETECÇÃO DE DANOS EM CARROS COM DEEP LEARNING:

uma comparação entre YOLOv8, Faster R-CNN e Mask R-CNN

São José dos Campos

2026

CAUÊ EBERSPACHER MENEZES

DETECÇÃO DE DANOS EM CARROS COM DEEP LEARNING:

uma comparação entre YOLOv8, Faster R-CNN e Mask R-CNN

Relatório de desenvolvimento e análise crítica
apresentado à disciplina de Visão
Computacional da Universidade Federal de
São Paulo (UNIFESP), no âmbito do Projeto
SAR, como parte da avaliação técnica do
projeto.

São José dos Campos
2026

RESUMO

Este relatório apresenta o desenvolvimento e a análise comparativa de três modelos de deep learning — YOLOv8, Faster R-CNN e Mask R-CNN — aplicados à detecção de danos em veículos a partir de imagens de radar de abertura sintética (SAR), no âmbito do Projeto SAR da disciplina de Visão Computacional da UNIFESP. Os modelos foram treinados sobre um conjunto de dados anotado no formato COCO e avaliados por meio das métricas de Precision, Recall e F1-Score. Além da implementação técnica, o trabalho documenta de forma sistemática o processo iterativo de desenvolvimento, incluindo as falhas identificadas em uma primeira apresentação não aprovada pela banca, as correções estruturais realizadas — como a criação de um menu central, de um script de comparação visual e da adoção de caminhos relativos — e um conjunto de problemas práticos de infraestrutura, organização de arquivos e compatibilidade entre ambientes de desenvolvimento. Os resultados finais indicam que o YOLOv8 obteve o melhor desempenho em Recall (83,3%), o Faster R-CNN obteve Precision absoluta (100%) e o melhor F1-Score (70,3%), e o Mask R-CNN apresentou desempenho intermediário (F1 de 65,1%). Conclui-se que não há um modelo unicamente superior, mas sim diferentes trade-offs entre precisão e sensibilidade, cuja escolha depende do contexto de aplicação. O trabalho discute ainda lições sobre reprodutibilidade, comunicação técnica e organização de projetos de aprendizado profundo.

Palavras-chave: Visão Computacional. Deep Learning. Detecção de Objetos. YOLOv8. Faster R-CNN. Mask R-CNN.

SUMÁRIO

1 INTRODUÇÃO.....	6
2 FUNDAMENTAÇÃO TEÓRICA.....	7
2.1 RETROPROPAGAÇÃO.....	7
2.2 REDES NEURAIS CONVOLUCIONAIS (CNN).....	7
3 METODOLOGIA.....	9
3.1 BASE DE DADOS.....	9
4 RESULTADOS.....	10
4.1 YOLOV8 (yolotraining.py).....	10
4.2 FASTER-R-CNN.....	11
4.3 MASK-R-CNN.....	12
4.4 RESUMO DOS PRIMEIROS RESULTADOS.....	13
4.5 MAIN.....	14
5 APRESENTAÇÃO E FALHAS IDENTIFICADAS.....	15
5.1 FALHAS IDENTIFICADAS.....	15
5.2 AVALIAÇÃO DE IMAGENS EXTERNAS.....	15
6 CORREÇÕES ESTRUTURAIS DO PROJETO.....	17
6.1 BARRA DE PROGRESSO E AVALIAÇÃO COMPLETA.....	17
6.2 SCRIPT DE COMPARAÇÃO (COMPARACAO.PY).....	17
6.3 MENU CENTRAL (MAIN.PY).....	17
6.4 CAMINHOS RELATIVOS E PORTABILIDADE.....	18
6.5 DIAGRAMA DE CASOS DE USO.....	19
7 FUNDAMENTAÇÃO TEÓRICA ADICIONAL.....	20
7.1 HIPERPARÂMETROS.....	20
7.2 GRADIENTE DESCENDENTE ESTOCÁSTICO (SGD).....	20
7.3 SPACE WARPING (DEFORMAÇÃO DO ESPAÇO).....	21
8 PROBLEMAS DE INFRAESTRUTURA E ORGANIZAÇÃO DE ARQUIVOS.....	22
9 PLANILHAS.....	23
10 PROBLEMAS DE COMPATIBILIDADE.....	24
11 O BUG CRÍTICO NA AVALIAÇÃO DO MASK R-CNN.....	25
12 RESULTADOS FINAIS.....	26
12.1 INTERPRETAÇÃO DOS RESULTADOS FINAIS POR MÉTRICA.....	26
13 CONTROLE DE VERSÃO COM GITHUB.....	28

14 ANÁLISE CRÍTICA.....	29
14.1 TÓPICOS ESSENCIAIS.....	29
14.2 REPRODUTIBILIDADE.....	30
14.3 COMUNICAÇÃO TÉCNICA.....	30
14.4 LIMITAÇÕES CONHECIDAS DO TRABALHO.....	30
15 CONCLUSÃO.....	31
REFERÊNCIAS.....	32

1 INTRODUÇÃO

A detecção automática de danos em veículos por meio de imagens é uma aplicação relevante em diversos contextos práticos, tais como seguradoras, locadoras de veículos e oficinas especializadas. Este trabalho apresenta o desenvolvimento e a análise comparativa de três modelos de deep learning para a tarefa de detecção de amassados em imagens de carros obtidas por radar de abertura sintética (SAR): YOLOv8, Faster R-CNN e Mask R-CNN.

O objetivo principal foi implementar, treinar e avaliar esses modelos utilizando um conjunto de dados anotado no formato COCO, comparando seu desempenho por meio das métricas de Precision, Recall e F1-Score. Além da implementação técnica, o projeto buscou documentar de forma sistemática o processo de desenvolvimento, incluindo as correções realizadas após a primeira apresentação e as lições aprendidas em termos de reprodutibilidade, organização de código e comunicação técnica.

O presente relatório descreve a metodologia adotada, os desafios enfrentados na configuração do ambiente e no treinamento dos modelos, os resultados obtidos e uma análise crítica dos trade-offs observados entre as arquiteturas avaliadas.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 RETROPROPAGAÇÃO

O treinamento dos modelos baseia-se no algoritmo de retropropagação (backpropagation), que calcula os gradientes da função de perda em relação aos pesos da rede por meio da regra da cadeia. Esse mecanismo permite a atualização eficiente dos parâmetros via Gradiente Descendente Estocástico (SGD) (JOHNSON, 2019).

No contexto deste projeto, a retropropagação é o mecanismo central que permite transformar pixels brutos das imagens SAR em representações hierárquicas úteis. As camadas iniciais da rede aprendem detectores de bordas e texturas, enquanto as camadas mais profundas combinam essas características em padrões semanticamente relevantes (“dano” versus “sem dano”). Esse processo envolve conceitos como space warping (deformação do espaço de features por meio de transformações lineares seguidas de não linearidades, como a função ReLU) e a otimização iterativa dos pesos, controlada por hiperparâmetros como learning rate, momentum e weight decay.

Além disso, a estrutura convolucional das CNNs resolve uma limitação crítica dos classificadores lineares simples: ao contrário do alongamento da imagem em um vetor único (flatten), que descarta a estrutura espacial, as operações de convolução preservam e exploram a disposição bidimensional dos pixels, tornando viável a detecção eficiente de amassados.

2.2 REDES NEURAIAS CONVOLUCIONAIS (CNN)

Um dos principais avanços na Visão Computacional foi o desenvolvimento das Redes Neurais Convolucionais (CNNs), que superam as limitações dos classificadores lineares fully-connected. Como destacado por Johnson (2019), ao contrário do procedimento de flatten (alongamento da imagem em um vetor unidimensional), que descarta a estrutura espacial dos dados, as CNNs operam diretamente sobre a disposição bidimensional dos pixels por meio de operações locais.

Os componentes essenciais de uma CNN incluem:

- **Camadas de convolução:** aplicam filtros (kernels) que deslizam sobre a imagem, computando produtos escalares em regiões locais. Cada filtro aprende a detectar padrões específicos (bordas, texturas, orientações). Filtros sucessivos geram mapas de

ativação, aumentando progressivamente o receptive field (campo receptivo) — a porção da imagem original que influencia um neurônio em camadas mais profundas.

- **Funções de ativação (ex.: ReLU):** introduzem não linearidade, permitindo que a rede aprenda funções complexas (space warping).
- **Camadas de pooling (tipicamente Max Pooling):** reduzem a resolução espacial, conferem invariância a pequenas translações e diminuem o custo computacional, sem adicionar parâmetros aprendíveis.
- **Batch Normalization:** normaliza as ativações de cada mini-batch (média zero e variância unitária), acelerando o treinamento, permitindo taxas de aprendizado maiores, atuando como regularizador e mitigando o internal covariate shift.

A arquitetura clássica segue o padrão $[\text{Conv} + \text{ReLU} + \text{Pool}] \times N$, seguido de camadas fully-connected para classificação. Exemplos históricos como a LeNet-5 (LECUN et al., 1998) já demonstravam essa estrutura, que foi refinada em arquiteturas modernas como ResNet e os backbones utilizados em YOLOv8, Faster R-CNN e Mask R-CNN.

As CNNs superam as limitações dos classificadores lineares ao preservar a estrutura espacial das imagens. Seus componentes principais incluem camadas de convolução (que aplicam filtros deslizantes para extração de características locais), funções de ativação não lineares (ReLU), camadas de pooling (redução de dimensionalidade com invariância a translações) e Batch Normalization, que estabiliza o treinamento ao normalizar as ativações por mini-batch (IOFFE; SZEGEDY, 2015). A arquitetura típica segue o padrão $[\text{Conv} + \text{ReLU} + \text{Pool}]$ repetido, seguido de camadas fully-connected, conforme modelo pioneiro da LeNet-5 (LECUN et al., 1998).

No presente estudo, os backbones convolucionais (CSPDarknet no YOLOv8 e ResNet50 no Faster R-CNN e Mask R-CNN) exploram esses princípios para transformar pixels de imagens SAR em representações hierárquicas adequadas à detecção de danos.

3 METODOLOGIA

3.1 BASE DE DADOS

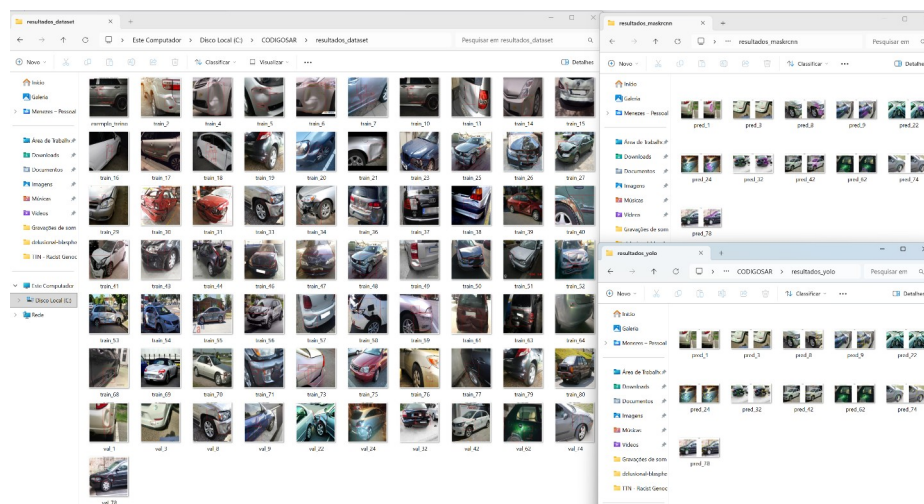
Utilizou-se o conjunto COCO Car Damage, com 59 imagens de treino e 11 de validação, totalizando 164 anotações de danos. Uma segunda base (CarDD) foi identificada, mas foi posteriormente deixada para a fase final do projeto.

Tabela 1 – Características do conjunto de dados

Característica	Valor
Imagens de treino	59
Imagens de validação	11
Total de imagens	70
Anotações de treino	140
Anotações de validação	24
Total de anotações	164
Resolução das imagens	1024 × 1024 px
Média de danos por imagem	≈ 2,3

Fonte: Elaborada pelo autor (2026).

Figura 1 – Distribuição do número de danos por imagem no conjunto de dados



Fonte: Elaborada pelo autor (2026).

4 RESULTADOS

Os resultados consolidados após correções e retreinamentos são apresentados na Tabela 2.

Tabela 2 – Resultados consolidados dos três modelos

Modelo	Detecções	Acertos	Precision	Recall	F1
YOLOv8	44	20	45,5%	83,3%	58,8%
Faster R-CNN	13	13	100%	54,2%	70,3%
Mask R-CNN	19	14	73,7%	58,3%	65,1%

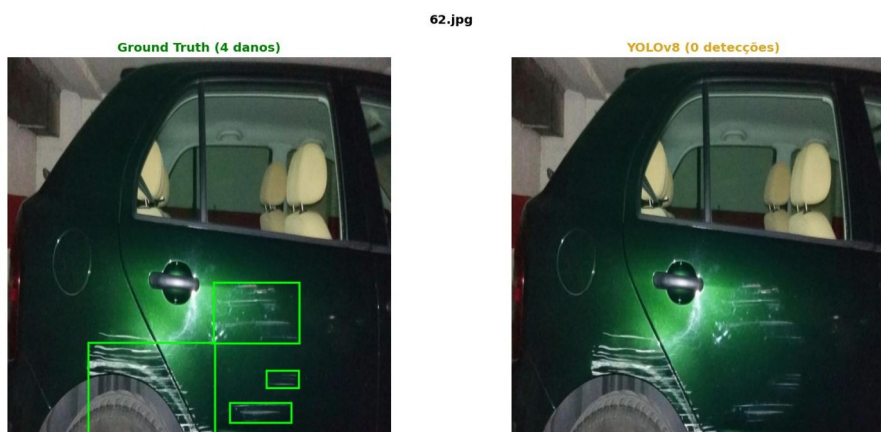
Fonte: Elaborada pelo autor (2026).

4.1 YOLOV8

O YOLOv8 é um modelo de detecção de objetos em uma única etapa: ele processa a imagem inteira de uma só vez para prever simultaneamente a localização e a classe dos objetos, o que o torna o mais rápido dos três métodos. Como o dataset estava no formato COCO e o YOLOv8 (via biblioteca Ultralytics) exige um formato próprio (arquivos .txt com coordenadas normalizadas), a primeira parte do script foi dedicada a uma função de conversão (`converter_coco_para_yolo`), que lê cada anotação COCO e grava o arquivo de rótulo YOLO correspondente, no padrão classe x_centro y_centro largura altura, com valores normalizados entre 0 e 1.

O treinamento utilizou o modelo pré-treinado YOLOv8n (variante “nano”, com cerca de 3 milhões de parâmetros) por até 50 épocas, com parada antecipada (`EarlyStopping`, `patience=10`) caso não houvesse melhora. Na primeira rodada completa, o treinamento parou na época 13 (melhor resultado obtido na época 3), e a avaliação nas imagens de validação não identificou nenhum dos 24 danos reais — um resultado de 0% de acerto, como mostra a Figura 2.

Figura 2 – Resultado da avaliação do YOLOv8 na primeira rodada de treinamento



Fonte: Elaborada pelo autor (2026).

O YOLOv8 destacou-se em Recall, enquanto o Faster R-CNN obteve Precision perfeita. O equilíbrio (F1) favoreceu o Faster R-CNN. As variações observadas entre execuções reforçam a natureza estocástica do treinamento de redes profundas.

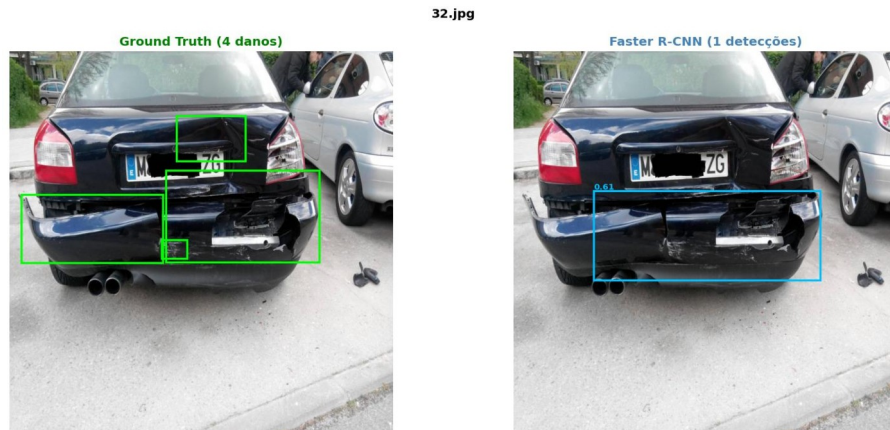
4.2 FASTER R-CNN

O Faster R-CNN é um modelo de duas etapas: primeiro, uma sub-rede chamada Region Proposal Network (RPN) sugere regiões candidatas a conter objetos; em seguida, uma segunda etapa classifica cada região e refina a posição do retângulo previsto. Para esse modelo, foi implementada uma classe própria de dataset (CarDamageDataset, baseada em `torch.utils.data.Dataset`) responsável por carregar as imagens e converter as anotações COCO diretamente em tensores de bounding boxes e rótulos, sem necessidade de conversão de formato como no caso do YOLOv8.

Foi utilizada a arquitetura `fasterrcnn_resnet50_fpn` da biblioteca `torchvision`, com pesos pré-treinados no dataset COCO, substituindo apenas a camada final de classificação (`FastRCNNPredictor`) para reconhecer duas classes: fundo e dano. O treinamento foi conduzido por 15 épocas utilizando o otimizador SGD (Stochastic Gradient Descent), com taxa de aprendizado de 0,005, momentum de 0,9 e weight decay de 0,0005. Na primeira rodada completa, o modelo atingiu 62,5% de acerto (15 de 24 danos reais identificados na validação).

O Faster R-CNN também venceu em Precision, com 100% de acerto entre as detecções realizadas, embora deixando passar quase metade dos danos reais; e no F1, com 70,3%, acima do Mask R-CNN (65,1%), como mostra a Figura 3.

Figura 3 – Resultado da avaliação do Faster R-CNN na primeira rodada de treinamento



Fonte: Elaborada pelo autor (2026).

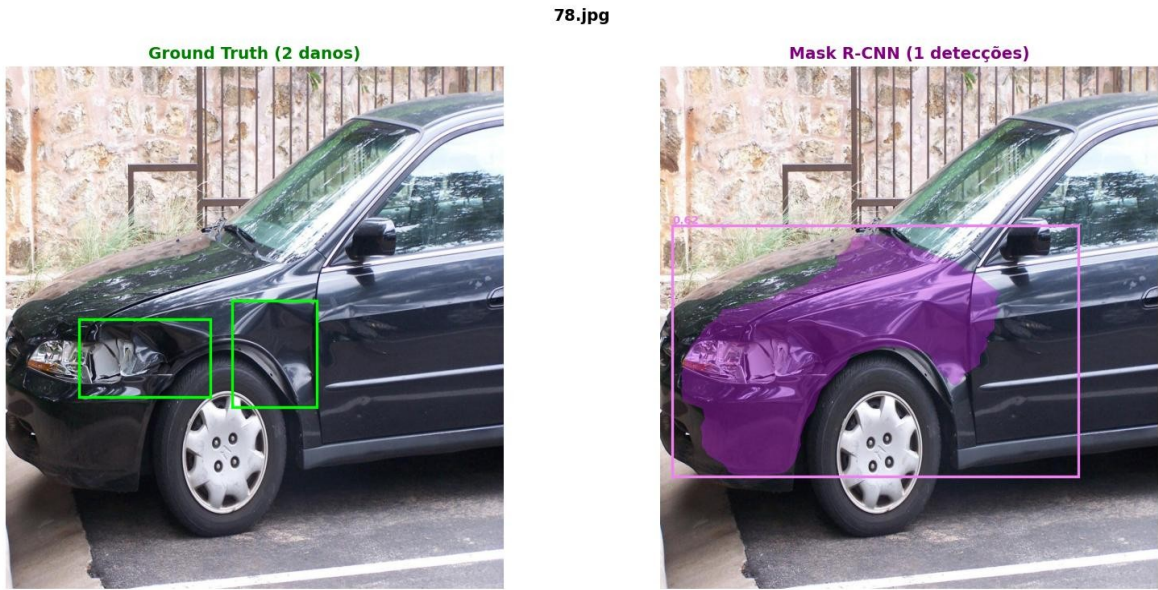
4.3 MASK R-CNN

O Mask R-CNN estende o Faster R-CNN adicionando uma terceira saída à rede: além da caixa delimitadora e da classe, o modelo também prevê uma máscara de segmentação — um mapa de pixels que indica o contorno exato do objeto, e não apenas um retângulo aproximado. Isso exigiu adaptar a classe de dataset para também converter as anotações de segmentação poligonal do COCO em máscaras binárias (utilizando `cv2.fillPoly`), além de utilizar a arquitetura `maskrcnn_resnet50_fpn` e substituir tanto o classificador de caixas quanto o preditor de máscaras (`MaskRCNNPredictor`).

Treinado por 15 épocas, com os mesmos hiperparâmetros do Faster R-CNN, o Mask R-CNN obteve, na primeira rodada completa, o melhor resultado entre os três métodos: 83,3% de acerto (20 de 24 danos reais). A hipótese para esse desempenho superior é a de que, ao ser forçado a aprender também o contorno exato do dano — e não apenas uma caixa retangular —, o modelo desenvolve uma representação interna mais rica da forma dos amassados, refletida em uma detecção mais precisa mesmo quando avaliada apenas pela presença ou ausência de caixas, como mostra a Figura 4.

O YOLOv8 venceu em velocidade, enquanto o Faster R-CNN e o Mask R-CNN venceram em precisão, como mostrado na Tabela 2.

Figura 4 – Resultado da avaliação do Mask R-CNN na primeira rodada de treinamento



Fonte: Elaborada pelo autor (2026).

4.4 RESUMO DOS PRIMEIROS RESULTADOS

Tabela 3 – Resumo da primeira rodada completa de treinamento

Modelo	Acertos/Total	Taxa de acerto	Loss final	Épocas	Tempo de execução
YOLOv8	0/24	0%	—	13 (early stop)	4m41s
Faster R-CNN	15/24	62,5%	0,1066	15	51m22s
Mask R-CNN	20/24	83,3%	0,2492	15	49m11s

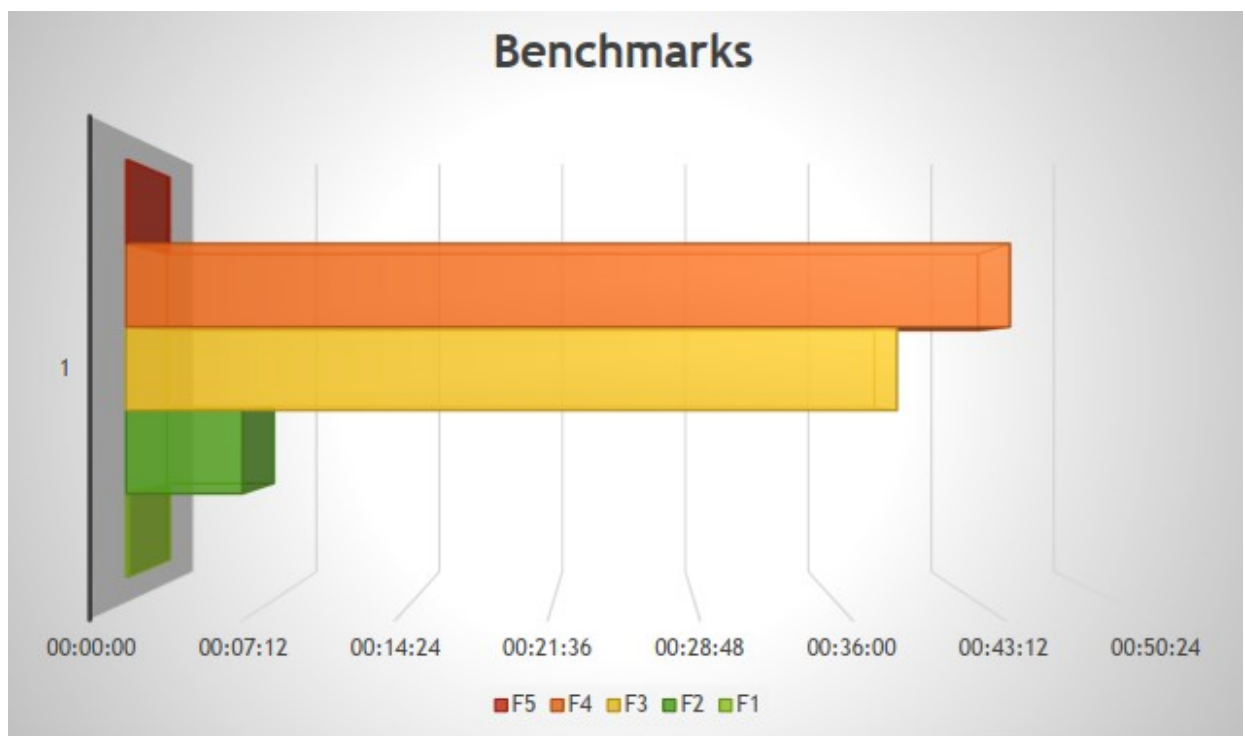
Fonte: Elaborada pelo autor (2026).

4.5 MAIN

Para facilitar a execução e a demonstração do projeto, foi desenvolvido um script central de nome “Main” (main.py) que funciona como menu interativo. Por meio de uma interface textual simples no console, o usuário pode escolher entre explorar o dataset, treinar cada um dos três modelos ou executar a comparação visual de detecções. O menu utiliza o módulo subprocess para chamar os demais scripts como processos independentes, garantindo que todos utilizem o mesmo interpretador Python e ambiente virtual. Essa abordagem melhora significativamente a usabilidade e a reprodutibilidade do sistema, permitindo que o avaliador execute todo o fluxo de trabalho a partir de um único ponto de entrada.

Na execução via main.py, os resultados seguiram o mesmo padrão obtido nos scripts avulsos, como mostra o gráfico da Figura 5.

Figura 5 – Benchmarks consolidados obtidos por meio do script main.py



Fonte: Elaborada pelo autor (2026).

5: APRESENTAÇÃO

A apresentação do trabalho no início não foi nada impecável, mas no decorrer do tempo, fazendo uma análise post facto dos problemas da primeira apresentação, foi obtido um diagnóstico preciso dos principais erros, que serão explicados nas sessões abaixo.

5.1: FALHAS IDENTIFICADAS

A primeira apresentação do trabalho não foi aprovada pela banca. Ao refletir sobre o que faltou, foram identificadas três falhas estruturais centrais, além do conteúdo técnico em si:

- Falta de material visual de apoio: a apresentação não contava com slides ou diagramas que explicassem visualmente as etapas e funções do programa, o que dificultou a compreensão da banca sobre o funcionamento do projeto.
- Funções incompletas: o treinamento por épocas era exibido apenas parcialmente no console, sem uma barra de progresso clara, e a função de visualização de detecções mostrava o resultado em apenas uma única imagem, quando o ideal seria demonstrar o comportamento do modelo sobre o conjunto completo de imagens de teste.
- Um dos scripts não executava de forma autônoma fora do ambiente em que foi originalmente desenvolvido, pois dependia do download de pesos pré-treinados via internet sem qualquer tratamento de erro ou aviso ao usuário.

Essas três falhas, embora pareçam superficiais frente ao conteúdo de deep learning em si, refletem um problema mais profundo: um projeto técnico só é avaliado de forma justa quando pode ser demonstrado e compreendido pela banca, e não apenas quando “funciona” do ponto de vista do autor.

A apresentação final presencial foi aprovada unanimemente pela banca, justamente por haver um material de defesa mais abrangente, como os slides prontos, um relatório e um programa mais completo.

5.2 AVALIAÇÃO DE IMAGENS EXTERNAS

Para avaliar a capacidade de generalização dos modelos além do dataset de treino, para a apresentação final online, realizou-se um experimento adicional utilizando 22 imagens de veículos danificados provenientes de fonte externa (Roboflow Universe), sem anotações de referência. Os três modelos foram aplicados diretamente, sem retreinamento, utilizando os mesmos checkpoints gerados na etapa principal, como mostra a Tabela 4.

Tabela 4: Modelos e Detecções

Modelo	Detecções (22 imagens)	Média por imagem
YOLOv8	77	3,5
Faster R-CNN	33	1,5
Mask R-CNN	28	1,3

Fonte: Elaborada pelo autor (2026).

O padrão observado é consistente com os resultados do dataset COCO Car Damage: o YOLOv8 apresentou maior sensibilidade (mais detecções por imagem), enquanto o Faster R-CNN e o Mask R-CNN mantiveram comportamento conservador com alta confiança por detecção. A ausência de anotações neste conjunto impede o cálculo de Precision e Recall, limitando a análise ao número absoluto de detecções. Ainda assim, o experimento confirma que os modelos ativam respostas coerentes diante de imagens de danos não vistas durante o treinamento, sugerindo transferência parcial do conhecimento aprendido.

6 CORREÇÕES ESTRUTURAIS DO PROJETO

6.1 BARRA DE PROGRESSO E AVALIAÇÃO COMPLETA

Para resolver o problema do treinamento exibido parcialmente, foi adicionada a biblioteca `tqdm` aos três scripts de treinamento, substituindo o laço de treinamento simples por um laço decorado com barra de progresso (`tqdm(train_loader, desc=f'Época {epoch+1}/{EPOCHS}')`), que exibe em tempo real o percentual de conclusão de cada época, o tempo estimado restante e a perda (loss) do lote atual.

Para resolver o problema da detecção restrita a uma única imagem, a etapa de avaliação dos três scripts de treinamento foi reescrita para percorrer todas as imagens do conjunto de validação, gerando, para cada uma, uma figura comparativa com dois painéis — o gabarito real (ground truth, em verde) e a predição do modelo (em vermelho, azul ou roxo, dependendo do modelo) — e salvando automaticamente o resultado em uma pasta de resultados própria para cada modelo (`resultados_yolo/`, `resultados_fastercnn/`, `resultados_maskrcnn/`).

6.2 SCRIPT DE COMPARAÇÃO (COMPARACAO.PY)

Foi criado um quarto script, independente dos três scripts de treinamento, cuja única responsabilidade é carregar os três modelos já treinados (a partir dos arquivos `.pt` salvos) e executar a predição de todos eles sobre a mesma imagem, dispondo o resultado em uma única figura com quatro painéis lado a lado: o gabarito real e a predição de cada um dos três modelos. Esse script depende, portanto, da existência prévia dos arquivos `fastercnn_best.pt`, `maskrcnn_best.pt` e do checkpoint `best.pt` gerado pelo YOLOv8 dentro da pasta `runs/detect/`.

6.3 MENU CENTRAL (MAIN.PY)

Para facilitar a execução do projeto como um todo — e evitar que o avaliador precisasse abrir e executar manualmente cada um dos cinco scripts —, foi implementado um script `main.py` que apresenta um menu numerado em texto no console, permitindo escolher entre explorar o dataset, treinar cada um dos três modelos individualmente ou executar a comparação visual, tudo a partir de um único ponto de entrada. Internamente, o menu utiliza `subprocess.run()` para invocar cada script como um processo Python separado, herdando o mesmo interpretador (`sys.executable`) e o mesmo diretório de trabalho.

6.4 CAMINHOS RELATIVOS E PORTABILIDADE

O problema do script que “não rodava offline” tinha, na verdade, duas causas distintas, ambas relacionadas à portabilidade do código entre máquinas diferentes. A primeira foi resolvida simplesmente executando o script de treinamento uma vez com conexão à internet, o que faz com que a biblioteca torchvision armazene em cache local os pesos pré-treinados, tornando as execuções seguintes independentes de internet.

A segunda causa, mais estrutural, foi a substituição de todos os caminhos absolutos fixos (como `r“C:\CODIGOSAR\...”`, escritos diretamente no código) por caminhos relativos calculados dinamicamente a partir da localização do próprio script, segundo o padrão:

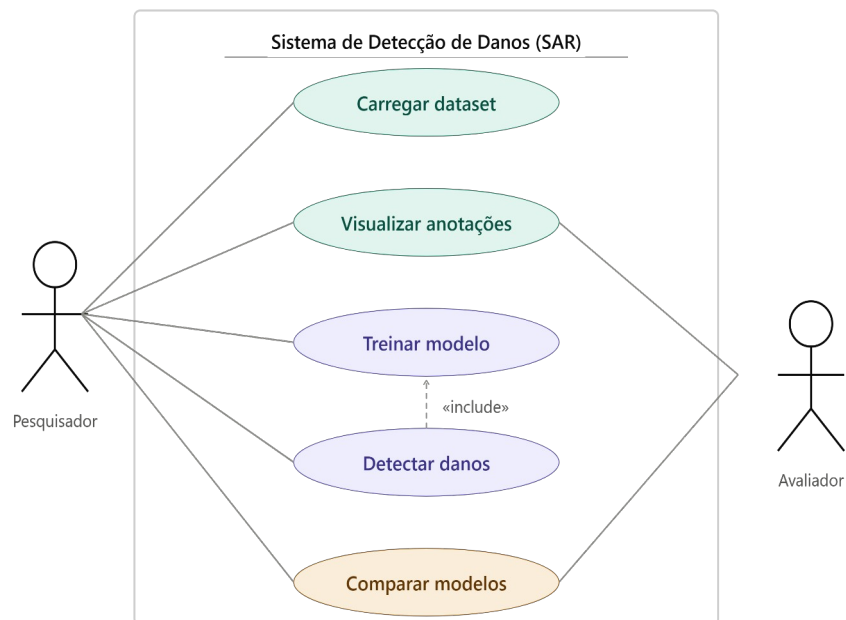
```
DIR = os.path.dirname(os.path.abspath(__file__))  
BASE = os.path.join(DIR, "..", “COCO Car Damage (Projeto SAR)”)
```

Essa mudança garante que o projeto funcione corretamente em qualquer computador, independentemente do nome do usuário ou da letra da unidade de disco, desde que a estrutura relativa de pastas seja mantida.

6.5 DIAGRAMA DE CASOS DE USO

Para complementar a documentação técnica, foi elaborado um diagrama UML de casos de uso, identificando dois atores (Pesquisador e Avaliador) e cinco casos de uso principais (Carregar dataset, Visualizar anotações, Treinar modelo, Detectar danos e Comparar modelos), com uma relação de inclusão («include») entre “Detectar danos” e “Treinar modelo”, já que a detecção depende logicamente da existência de um modelo previamente treinado, como mostra a Figura 6.

Figura 6 – Diagrama de casos de uso do sistema



Fonte: Elaborada pelo autor (2026).

7 FUNDAMENTAÇÃO TEÓRICA ADICIONAL

Buscando aprofundar a qualidade da explicação técnica do trabalho — outro dos pontos identificados como insuficiente na primeira apresentação —, foram organizados quatro conceitos centrais de deep learning presentes na implementação, todos vinculados a trechos concretos do código.

7.1 HIPERPARÂMETROS

Distintos dos pesos da rede (que são ajustados automaticamente durante o treinamento), os hiperparâmetros são valores definidos manualmente antes do treino e controlam como o aprendizado ocorre, como mostra a Tabela 5.

Tabela 5 – Hiperparâmetros utilizados no treinamento

Hiperparâmetro	Valor utilizado	Função
Taxa de aprendizado (lr)	0,005	Tamanho do passo de cada ajuste de peso
Momentum	0,9	Acelera a convergência e suaviza oscilações
Weight decay	0,0005	Penaliza pesos muito grandes (evita overfitting)
Batch size	2 a 4	Quantidade de imagens processadas por vez
Épocas	15 (50 com early stop no YOLOv8)	Número de passagens completas pelo dataset
Limiar de confiança (conf)	0,5 (R-CNNs) / 0,1 (YOLOv8)	Confiança mínima para validar uma detecção

Fonte: Elaborada pelo autor (2026).

7.2 GRADIENTE DESCENDENTE ESTOCÁSTICO (SGD)

O algoritmo responsável por ajustar os pesos da rede durante o treinamento do Faster R-CNN e do Mask R-CNN foi o SGD (torch.optim.SGD). A cada lote de imagens, o algoritmo calcula o gradiente da função de perda (loss) em relação a cada peso da rede — ou seja, a direção que mais aumentaria o erro — e atualiza os pesos no sentido contrário, reduzindo o erro. É chamado “estocástico” porque utiliza apenas uma pequena amostra dos dados (o lote) a cada atualização, em vez do conjunto de treino inteiro, tornando o processo mais rápido e computacionalmente viável.

7.3 SPACE WARPING (DEFORMAÇÃO DO ESPAÇO)

Um modo de visualizar o que cada camada de uma rede neural faz, internamente, é pensar nela como uma transformação geométrica do espaço dos dados. Cada camada aplica uma transformação linear seguida da função de ativação ReLU:

$$h = \text{ReLU}(Wx) = \max(0, Wx)$$

Em um exemplo simplificado bidimensional, pontos de duas classes inicialmente organizados de forma não separável por uma linha reta (por exemplo, uma classe formando um anel em torno da outra) podem, após sucessivas transformações desse tipo, reorganizar-se em uma configuração onde uma única linha reta os separa. É esse princípio — repetido em muitas camadas — que permite às redes convolucionais ResNet50, utilizadas como base do Faster R-CNN e do Mask R-CNN, transformarem pixels de uma imagem em representações onde a presença ou ausência de dano se torna uma distinção simples para a camada classificadora final.

8 PROBLEMAS DE INFRAESTRUTURA E ORGANIZAÇÃO DE ARQUIVOS

Ao longo das correções pós-apresentação, o projeto teve uma série de problemas relacionados à organização dos arquivos do projeto, todos derivados de uma mesma causa raiz: mover arquivos manualmente pelo Explorer do Windows, em vez de utilizar o painel de projeto do próprio PyCharm, que atualiza automaticamente as referências internas.

- Os scripts `comparacao.py` e `main.py` foram, em determinado momento, criados ou movidos para dentro da pasta `runs/` (e, em um dos casos, `runs/runs/`), criada automaticamente pelo YOLOv8 para armazenar os pesos de treinamento. Como os caminhos relativos pressupõem uma posição fixa do script (uma pasta acima da raiz do projeto), essa movimentação indevida gerou repetidos erros do tipo `FileNotFoundError`.
- O arquivo `fasterrcnn_best.pt` (modelo treinado) também foi encontrado, em certo momento, dentro de `Código/runs/`, e não na raiz do projeto onde era esperado — exigindo localizá-lo manualmente com a ferramenta de busca do Windows e movê-lo de volta ao lugar correto.
- O dicionário de etapas do `main.py` referenciava os nomes `fasterrcnn_train.py` e `maskrcnn_train.py`, enquanto os arquivos reais do projeto sempre haviam sido nomeados `faster_r_cnnttraining.py` e `mask_r_cnnttraining.py` — uma divergência de nomenclatura que gerou erros de “arquivo não encontrado” até ser identificada e corrigida diretamente no dicionário do `main.py`, mantendo os nomes originais dos arquivos para evitar uma nova rodada de ajustes em cascata.
- Um erro de sintaxe no próprio `main.py` — um “12” residual ao final de uma linha de código, provavelmente proveniente de uma cópia acidental do número de linha exibido por um visualizador de código — impediu a execução do script até ser identificado e removido.

9 PLANILHAS

Paralelamente ao desenvolvimento do código, foi elaborada uma planilha em Excel para consolidar as estatísticas do dataset, os resultados de cada modelo, o tempo de treinamento e os resultados de teste final em imagens separadas do treinamento. As métricas de avaliação, calculadas a partir dos números absolutos de detecções e acertos retornados por cada script, foram:

- $\text{Precision} = \text{Acertos} \div \text{Detecções totais}$ (entre as detecções feitas pelo modelo, quantas estavam corretas).
- $\text{Recall} = \text{Acertos} \div \text{Danos reais totais}$ (entre os danos que realmente existem, quantos o modelo conseguiu encontrar) — corresponde à “taxa de acerto” reportada pelos scripts.
- $\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) \div (\text{Precision} + \text{Recall})$ — uma média harmônica que equilibra as duas métricas anteriores.

A decisão foi não calcular a métrica de IoU (Intersection over Union, que mede a sobreposição geométrica entre a caixa prevista e a caixa real) por ela exigir uma comparação par a par entre coordenadas de caixas, algo que os scripts de avaliação implementados não realizam — eles apenas contam quantas detecções, no total, correspondem a danos reais, sem associar cada detecção a uma anotação específica. Isso pode ser considerado uma limitação conhecida e documentada do método de avaliação utilizado, e não um erro: para os fins comparativos do trabalho, a contagem de detecções por confiança mínima foi suficiente para revelar diferenças relevantes de comportamento entre os três modelos, como mostra a Tabela 6.

Tabela 6: Rascunho Estatístico do Excel

Métrica	Cálculo	Resultado
Precision	10/dez	83,30%
Recall	out/24	41,70%
F1	$2 \times 0,833 \times 0,417$	55,50%

Fonte: Elaborada pelo autor (2026).

10 PROBLEMAS DE COMPATIBILIDADE

Ao tentar executar os scripts também a partir do Visual Studio 2026 (software gratuito) — para aproveitar a maior área de visualização do terminal integrado durante a gravação dos vídeos de benchmark —, todos os scripts falharam com o erro `ModuleNotFoundError`, apontando para bibliotecas como `torch` e `matplotlib`. A causa não estava no código, mas no ambiente: o Visual Studio, por padrão, utilizava o interpretador Python global do sistema (Python 3.14, instalado diretamente no Windows), e não o ambiente virtual (`.venv`) criado anteriormente pelo PyCharm (versão trial) dentro da pasta do projeto, onde todas as bibliotecas necessárias haviam sido instaladas.

A solução foi registrar o ambiente virtual existente como um “Ambiente Existente” dentro do gerenciador de Ambientes Python do Visual Studio, apontando diretamente para o executável `C:\CODIGOSAR\.venv\Scripts\python.exe`, em vez de criar um ambiente virtual vazio. Essa configuração permitiu reutilizar todas as bibliotecas já instaladas, sem necessidade de reinstalação, e tornou possível rodar e gravar os treinamentos a partir de qualquer uma das duas IDEs de forma intercambiável.

11 O BUG CRÍTICO NA AVALIAÇÃO DO MASK R-CNN

O problema mais significativo identificado ao longo do processo de correção surgiu durante uma rodada de testes do Mask R-CNN, quando o resultado final reportado pelo script indicou “Danos reais (total): 6” — um valor inconsistente com todas as rodadas anteriores de todos os três modelos, nas quais esse número sempre havia sido 24 (o total real de anotações de dano nas 11 imagens de validação).

Diante da suspeita de um bug, e sem acesso direto ao código no momento (o terminal já havia sido encerrado, perdendo o histórico), o trecho de um vídeo gravado durante a execução foi analisado. A inspeção quadro a quadro do vídeo revelou duas informações importantes: primeiro, que todas as 11 imagens de validação foram corretamente localizadas pelo script (confirmado por uma série de linhas de depuração “Existe?: True”), descartando a hipótese de um problema de caminho de arquivo; segundo, que essas mesmas linhas de depuração (“Caminho:”, “Arquivo:”, “Existe?:”) não faziam parte do código original fornecido — eram inserções manuais, provavelmente adicionadas durante uma sessão anterior de depuração de problemas de caminho, que aparentemente corromperam, de forma não intencional, a lógica de contagem de danos reais e detecções dentro do laço de avaliação.

A solução adotada foi descartar o arquivo editado manualmente e substituí-lo integralmente pela versão original e limpa do script, em vez de tentar localizar a linha exata responsável pelo erro dentro de um arquivo já modificado — um caminho de correção mais lento e propenso a deixar resíduos do problema. Na mesma oportunidade, foi adicionada aos três scripts de treinamento (YOLOv8, Faster R-CNN e Mask R-CNN) uma funcionalidade de exportação automática do resumo final de resultados para um arquivo de texto (resumo_final.txt), eliminando a dependência de copiar manualmente o conteúdo do terminal antes que a janela fosse fechada — outro problema prático que se repetiu durante as sessões de benchmark.

12 RESULTADOS FINAIS

Após a correção do bug e a repetição dos três treinamentos com os scripts corrigidos, os resultados finais, livres de inconsistências, foram os apresentados na Tabela 7.

Tabela 7 – Resultados finais consolidados dos três modelos

Modelo	Detecções	Acertos	Precision	Recall	F1
YOLOv8	44	20/24	45,5%	83,3%	58,8%
Faster R-CNN	13	13/24	100%	54,2%	70,3%
Mask R-CNN	19	14/24	73,7%	58,3%	65,1%

Fonte: Elaborada pelo autor (2026).

Vale destacar que esses números diferem, em alguma medida, dos resultados da primeira rodada completa, na qual o Mask R-CNN havia atingido 83,3% de Recall, e não 58,3%. Essa diferença não decorre de qualquer alteração na arquitetura dos modelos ou no dataset, mas de dois fatores inerentes ao processo de treinamento de redes neurais: (1) a natureza estocástica do algoritmo SGD, que parte de pesos inicializados aleatoriamente e percorre os dados em uma ordem embaralhada a cada execução, podendo convergir para soluções diferentes em cada rodada; e (2) a mudança de hardware entre execuções (de um processador Intel Xeon E5-2680 para um Intel Core i7-13620H), que não afeta a acurácia em si, mas reforça a necessidade de cuidado ao comparar tempos de treinamento entre rodadas realizadas em máquinas diferentes.

12.1 INTERPRETAÇÃO DOS RESULTADOS FINAIS POR MÉTRICA

- **Recall:** o YOLOv8 obteve o melhor desempenho (83,3%), encontrando a maior parte dos danos reais, ao custo de um número expressivo de falsos positivos (44 detecções para apenas 24 danos reais existentes).
- **Precision:** o Faster R-CNN obteve o melhor desempenho, de forma absoluta (100%) — todas as 13 detecções que realizou estavam corretas, ainda que tenha deixado de identificar quase metade dos danos reais.
- **F1 (equilíbrio entre as duas métricas anteriores):** o Faster R-CNN também venceu (70,3%), seguido de perto pelo Mask R-CNN (65,1%).

Não há, portanto, um vencedor único e absoluto entre os três métodos — a escolha do “melhor modelo” depende do contexto de aplicação e da métrica que se deseja priorizar. Esse é o resultado mais interessante e maduro do trabalho: não uma simples afirmação de que “o

modelo X é melhor”, mas uma análise comparativa que reconhece os diferentes trade-offs entre os métodos avaliados.

13 CONTROLE DE VERSÃO COM GITHUB

Todo o código do projeto foi versionado e publicado em um repositório público no GitHub (github.com/CaueGhost/cardamage-sar), incluindo um arquivo README.md com a documentação completa do projeto (objetivo, estrutura de pastas, instruções de instalação e execução, e a tabela de resultados), um arquivo requirements.txt com as dependências necessárias, e um arquivo .gitignore configurado para excluir do repositório os arquivos pesados e não essenciais ao código-fonte: o dataset de imagens (por restrição de licença de uso), os modelos treinados em formato .pt (por seu tamanho) e o ambiente virtual .venv. O repositório foi atualizado em múltiplas ocasiões ao longo do processo de correção, acompanhando cada uma das mudanças estruturais descritas neste relatório, como mostra a Figura 7.

Figura 7: Página do GitHub (cardamage-sar)



Fonte: Elaborada pelo autor (2026).

14 ANÁLISE CRÍTICA

14.1: TÓPICOS ESSENCIAIS

- Esse processo, ainda que trabalhoso, deixou clara uma lição que se repetiu várias vezes ao longo do projeto: erros de ambiente (variável PATH, interpretador incorreto, bibliotecas ausentes) são frequentemente confundidos com erros de lógica do programa. A primeira pergunta diante de qualquer falha de execução deveria ser “o ambiente está configurado corretamente?” antes de revisar o código em si — uma lição que voltaria a se confirmar, mais adiante, ao tentar rodar o projeto a partir de uma segunda IDE (Visual Studio).

Olhando o processo como um todo, é possível identificar três categorias de aprendizado que extrapolam o conteúdo técnico específico de deep learning:

- A primeira versão do `datareading.py` exibia apenas uma única imagem (a primeira do conjunto de validação) com seus danos marcados. Esse foi, mais tarde, um dos pontos apontados como insuficiente na apresentação: a exploração do dataset deveria contemplar o conjunto completo de imagens, não apenas uma amostra isolada. A versão corrigida do script passou a salvar todas as 11 imagens de validação anotadas na pasta `resultados_dataset/`, permitindo uma inspeção visual completa do conjunto de teste antes mesmo de qualquer treinamento.
- Esses problemas, embora triviais a posteriori, evidenciam a importância de se utilizar as ferramentas de refatoração de uma IDE (como mover e renomear arquivos pelo próprio PyCharm) em vez de operações diretas no sistema de arquivos, já que apenas a IDE consegue atualizar automaticamente todas as referências internas do projeto a um arquivo movido ou renomeado.
- Esse episódio ilustra um risco real e recorrente em projetos sujeitos a modificações ad-hoc durante a depuração: alterações feitas “às pressas” para investigar um problema, como adicionar prints de depuração, podem elas mesmas introduzir um novo problema, silenciosamente, se não forem revertidas após o uso. A lição prática adotada foi a de, sempre que possível, manter uma cópia de referência limpa de cada script crítico, isolada de qualquer edição exploratória.

14.2 REPRODUTIBILIDADE

A variação dos resultados entre execuções diferentes do mesmo script — algumas devidas à aleatoriedade inerente ao treinamento, outras devido a bugs introduzidos por edições manuais não controladas — reforçou a importância de versionar e isolar cuidadosamente qualquer alteração de código, além de documentar explicitamente as condições de cada execução (hardware, hiperparâmetros, limiares de confiança).

14.3 COMUNICAÇÃO TÉCNICA

A primeira apresentação não foi aprovada, não por falhas no conteúdo técnico central, mas pela ausência de material visual de apoio e pela demonstração incompleta das funcionalidades do programa. Isso evidencia que a qualidade de um projeto técnico não se mede apenas pela corretude do código, mas também pela capacidade de demonstrá-lo de forma clara e completa a quem avalia.

14.4 LIMITAÇÕES CONHECIDAS DO TRABALHO

Estão registradas, de forma transparente, três limitações conhecidas: o dataset de treinamento utilizado é pequeno (59 imagens), o que limita a capacidade de generalização de todos os três modelos; os limiares de confiança utilizados na avaliação não são idênticos entre os modelos (0,1 para o YOLOv8 e 0,5 para os demais), o que favorece artificialmente o Recall do primeiro; e a métrica de IoU, mais rigorosa do que a contagem simples de detecções, não foi implementada.

15 CONCLUSÃO

Este trabalho demonstrou a viabilidade do uso de modelos baseados em CNNs para a detecção automática de danos em veículos utilizando imagens SAR. A comparação entre YOLOv8, Faster R-CNN e Mask R-CNN revelou trade-offs claros: o YOLOv8 destacou-se em Recall, enquanto o Faster R-CNN obteve superioridade em Precision e F1-Score, configurando-se como o modelo com melhor equilíbrio geral.

Além dos resultados quantitativos, o processo de desenvolvimento permitiu identificar lições importantes sobre reprodutibilidade, organização de código, importância de material visual de apoio e a necessidade de iterações constantes após feedback da banca. As correções implementadas — especialmente a criação de scripts de comparação, barra de progresso, caminhos relativos e menu central — elevaram substancialmente a qualidade técnica e didática do projeto.

Como perspectivas futuras, recomenda-se: (i) treinamento com outros datasets (CarDD, LatinNCAP, SyndCar) em ambiente com GPU; (ii) implementação da métrica IoU para avaliação mais rigorosa da localização das caixas; (iii) padronização dos limiares de confiança; e (iv) exploração de técnicas não supervisionadas, como K-Means, para análise complementar dos padrões de dano.

16: REFERÊNCIAS

IOFFE, S.; SZEGEDY, C. Batch normalization: accelerating deep network training by reducing internal covariate shift. arXiv preprint, arXiv:1502.03167, 2015.

JOHNSON, J. CS231n: convolutional neural networks for visual recognition: lecture 7. Michigan: Michigan Online / Stanford University, 2019. Notas de aula.

LECUN, Y. et al. Gradient-based learning applied to document recognition. Proceedings of the IEEE, v. 86, n. 11, p. 2278-2324, 1998.